отчет лабораторная работа 8

Ашихмин Кирилл Алексеевич 504639

Цель работы

1. Создать простое клиент-серверное приложение на Python без серверных фреймворков.
2. Освоить работу с HTTPServer и маршрутизацию запросов.
3. Применять шаблонизатор Jinja2 для отображения данных.
4. Реализовать модели предметной области (`User`, `Currency`, `UserCurrency`, `App`, `Author`) с геттерами и сеттерами.
5. Структурировать код в соответствии с архитектурой MVC.
6. Получать данные о курсах валют через функцию `get_currencies` и отображать их пользователям.
7. Реализовать функциональность подписки пользователей на валюты и отображение динамики их изменения.
8. Научиться создавать тесты для моделей и серверной логики.

Описание предметной области

Модели

1. **Author**
   - `name` — имя автора
   - `group` — учебная группа
2. **App**
   - `name` — название приложения
   - `version` — версия приложения
   - `author` — объект Author
3. **User**
   - `id` — уникальный идентификатор
   - `name` — имя пользователя
4. **Currency**
   - `id` — уникальный идентификатор
   - `num_code` — цифровой код
   - `char_code` — символьный код
   - `name` — название валюты

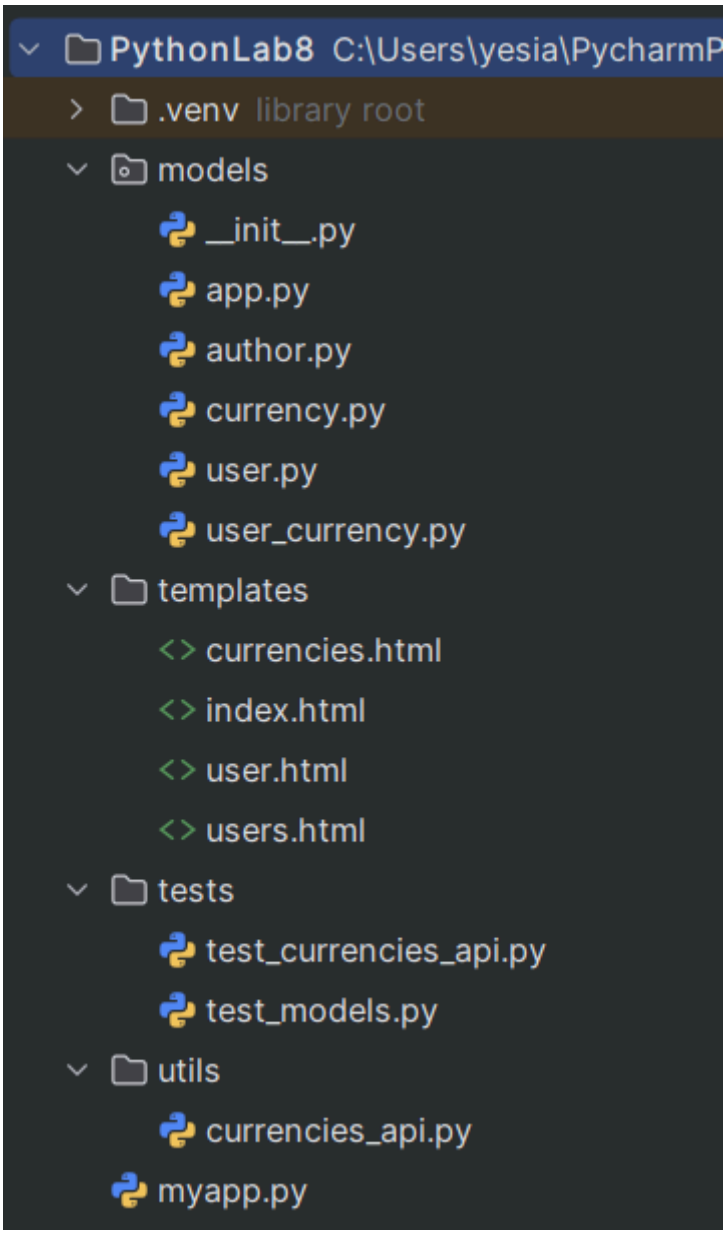
- `value` — курс
- `nominal` — номинал (за сколько единиц валюты указан курс)

Пример XML:

```
<Valute ID="R01280">
 <NumCode>360</NumCode>
 <CharCode>IDR</CharCode>
 <Nominal>10000</Nominal>
 <Name>Рупий</Name>
 <Value>48,6178</Value>
</Valute>
```

7. **UserCurrency**
    - `id` — уникальный идентификатор
    - `user_id` — внешний ключ к User
    - `currency_id` — внешний ключ к Currency
    - Реализует связь «много ко многим» между пользователями и валютами.

Структура проекта

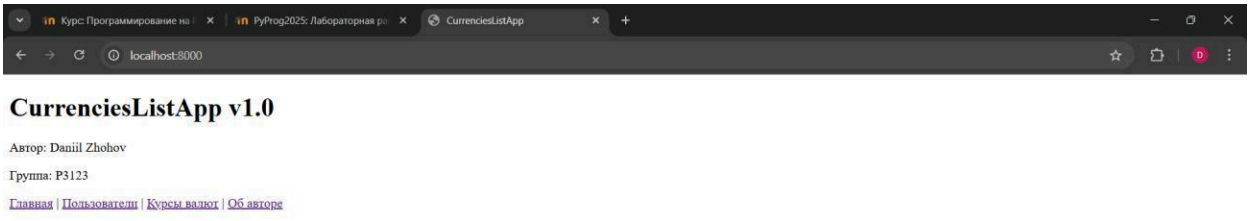


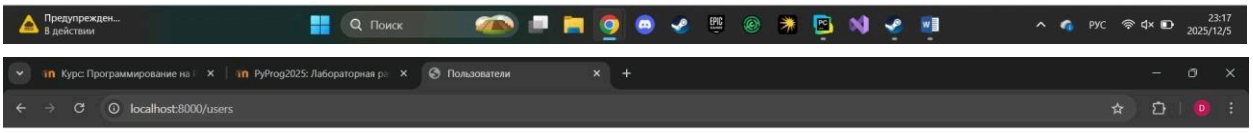
Описание реализации

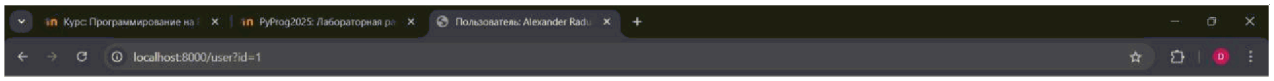
Все модели предметной области, включая автора, приложение, пользователя, валюту и связь между ними, оформлены в виде отдельных классов внутри специального пакета. Для каждого атрибута этих классов настроены механизмы получения и установки значений. При записи данных в поля объектов происходит строгая проверка: система контролирует тип данных, их непустоту и соответствие формату, например, длину символьного кода валюты. Если поступающее значение не соответствует требованиям, программа прерывает операцию и сообщает об ошибке, что защищает приложение от некорректного состояния. Уникальные идентификаторы для пользователей и связей генерируются автоматически с помощью внутренних счётчиков, имитируя работу базы данных.
Серверная часть построена на стандартных средствах языка без использования сторонних веб-фреймворков. Обработчик запросов анализирует адресную строку браузера и извлекает параметры. В зависимости от указанного пути сервер выбирает

необходимую логику: формирует набор данных, подключает соответствующий HTML-шаблон и отправляет готовую страницу пользователю с правильной кодировкой. Обработка адресов с параметрами, например для просмотра профиля конкретного пользователя, реализована вручную через разбор запроса.
Для отображения информации используется шаблонизатор, который настраивается единожды при запуске программы. Это позволяет хранить загруженные шаблоны в памяти для ускорения работы. Система автоматически экранирует опасные символы для безопасности. Данные передаются в шаблоны в виде именнованных переменных, что Keeps код обработки запросов чистым и независимым от вёрстки.
Получение актуальных курсов валют вынесено в отдельный utility-модуль. Программа отправляет запрос на официальный сайт Центрального банка, получает ответ в формате XML и разбирает его стандартными средствами. Особое внимание уделено преобразованию числовых данных, так как в источнике разделителем служит запятая, а программе требуется точка. Созданные объекты валют передаются дальше в систему для отображения на страницах списков и в личных кабинетах пользователей.

Примеры работы приложения







Honsaoøazeøs: Alexander Radulov

Подписки на валюты:

- USD (Доллар США): 76.0937 за 1

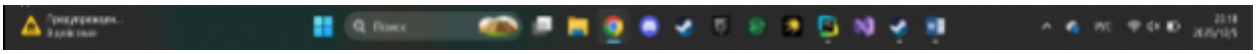
← Назад к списку

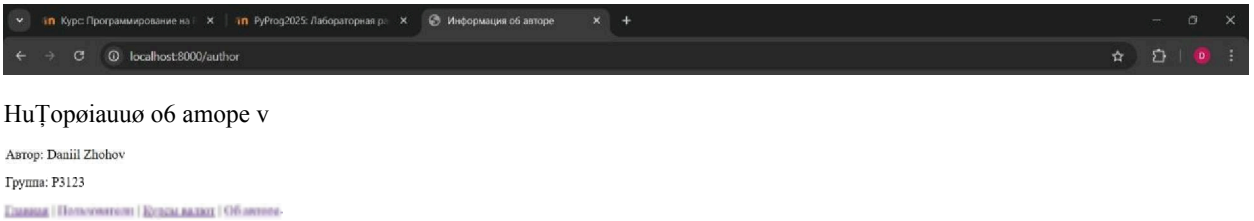
Honsaoøazens: Alexander Yelesin

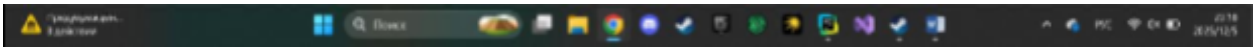
Подписки на валюты:

- CNY (Юань): 10.7328 за 1

← Назад к списку







Выводы

В процессе выполнения работы столкнулся с рядом технических трудностей. Initially возникли проблемы с подключением модулей внутри проекта из-за особенностей запуска главного скрипта, что решилось изменением способа импорта на абсолютный. При обработке данных от внешнего источника потребовалась дополнительная логика для замены разделителей в числах, так как формат источника отличался от ожидаемого программой. Отсутствие полноценной базы данных ограничило возможность сохранения состояния между перезапусками, однако для учебной задачи хватило хранения данных в памяти. Также реализация маршрутизации вручную потребовала больше усилий по разбору адресов по сравнению с использованием готовых фреймворков.

Архитектурный подход MVC был успешно внедрён в проект. Логика данных и правила их проверки сосредоточены в моделях и не зависят от способа отображения. Шаблоны отвечают только за визуальную часть и не содержат сложных вычислений. Управление запросами и связывание компонентов взя на себя контроллер. Такое разделение позволило вносить изменения в отдельные части программы, например добавлять графики, не переписывая базовые классы.

В результате были получены практические навыки создания веб-сервера на базовых библиотеках языка. Изучены принципы правильной настройки шаблонизатора для обеспечения производительности и безопасности. Приобретён опыт интеграции с внешними информационными системами, включая обработку сетевых ответов в формате XML и преобразование их во внутренние объекты приложения.